

datalex



Get Started with Databricks Data Science & Engineering Workspace



Alexandre BERGERE

Head of Data & AI Engineer & Partners at DataGalaxy, Data Architect independent
Delta & openLineage lover

Data Expert
(Audit, Modeling, Urbanization, BI's project – Azure, Big Data, Spark, openLineage & Delta lover)

Trainer
(BI, Big Data, MongoDB, NoSQL, SQL, Cloud Computing)

Speaker
(Pass Summit 2018, Spark AI Tour 2020, …)

Cloud & data tech articles
(medium, LinkedIn, slideshare)

Avanade
Sr Anls, Data Analyst

2016 - 2019

Trainer
2016 - x

ESAIP, ESME, ECS, NEOMA, ESEO, EFREI, Sorbonne Paris

BI, Big Data, MongoDB, NoSQL, Panorama de la data, Cloud computing

OpenClassroom
Mentor

2019 - x

Datalex
2019 - x

Data Architect Freelance

DataRedKite
CTO & Co-Founder

2020 - 2023

DataGalaxy
2021 - x

Head of Data & AI Engineer

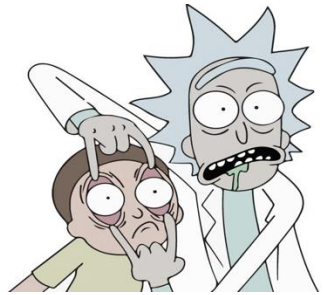
Don’t get lost



Practice



Demo

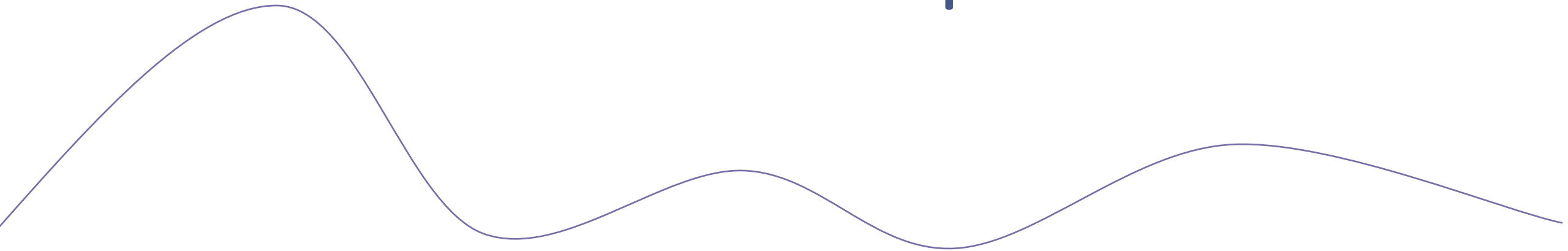


Important

Learning Objectives

1. Understand Spark
2. Understand the core components of the Databricks Lakehouse platform
3. Use PySpark in Azure Databricks

Understand Spark



Apache Spark

Apache Spark™ is a multi-language engine for executing data engineering, data science, and machine learning on single-node machines or clusters.

https://spark.apache.org/



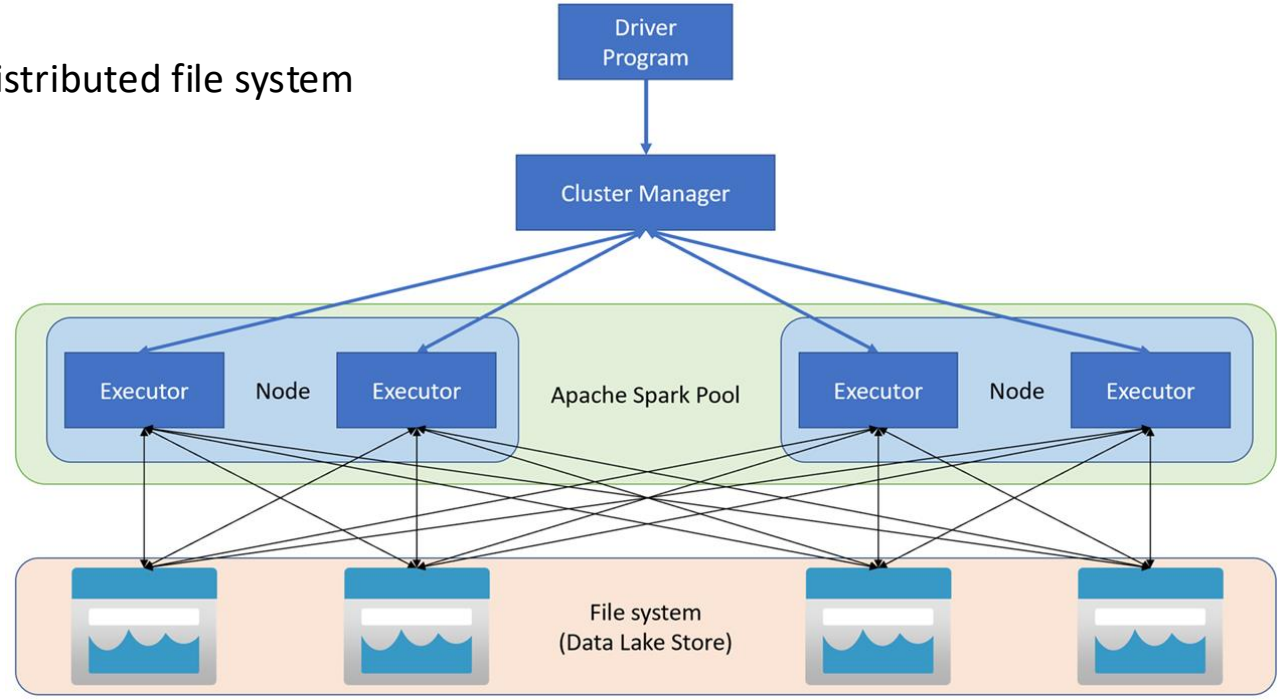
Forbes' article

Apache Spark

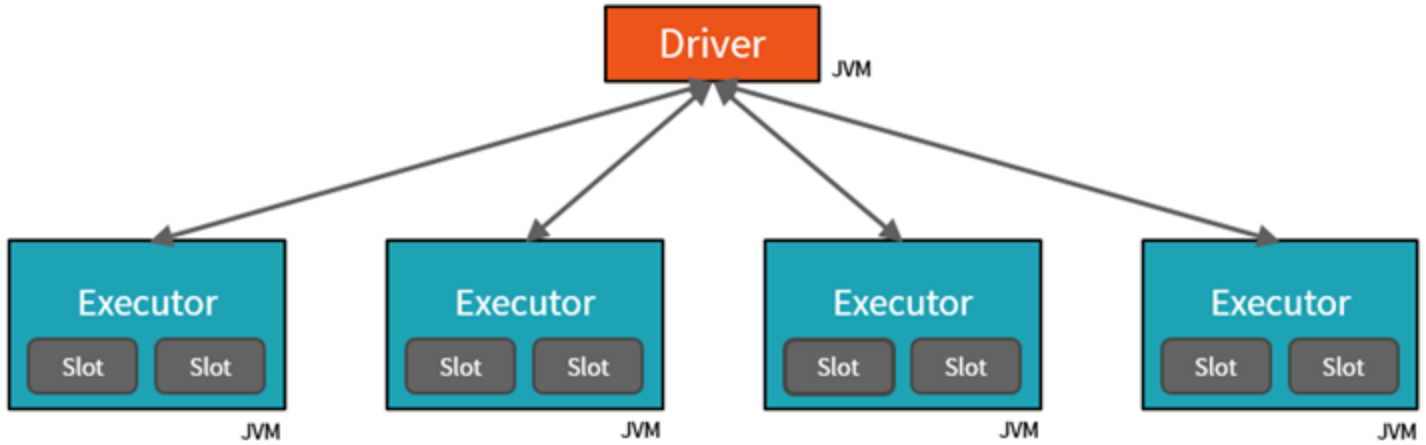
Distributed data processing framework

- Code in multiple languages
- Driver program uses SparkContext to coordinate processing across multiple executors on worker nodes
- Executors run in parallel to process data in a distributed file system



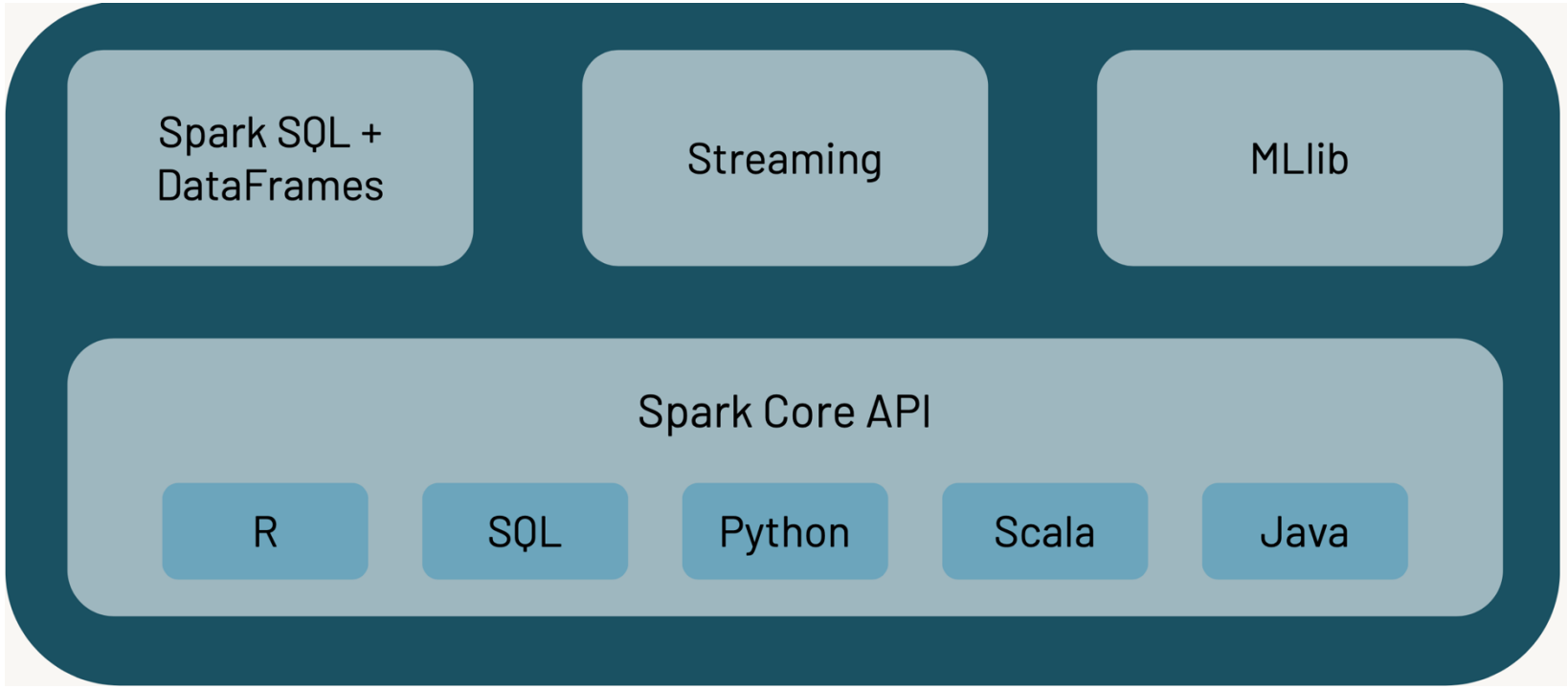


Apache Spark



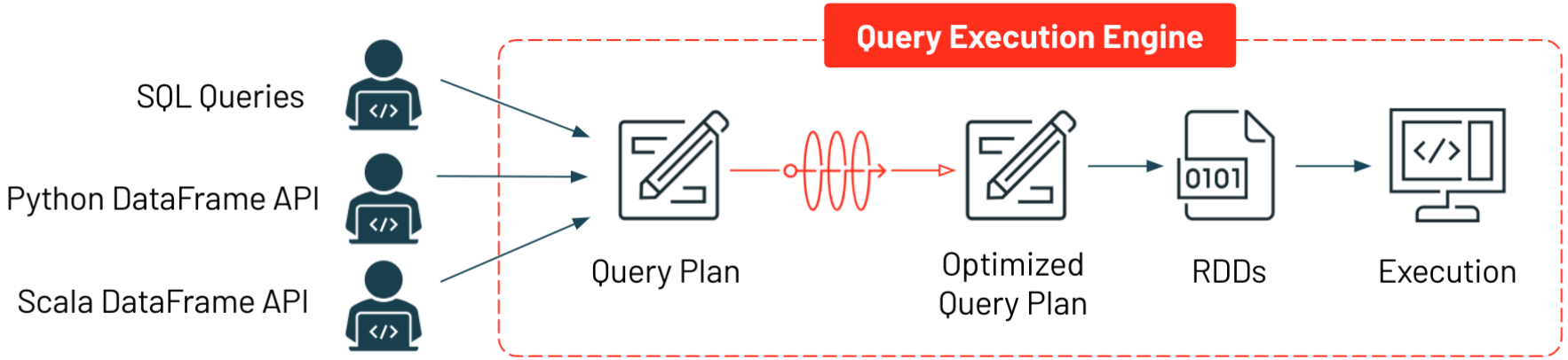
- The **Driver** is the JVM in which our application runs.
- The secret to Spark's awesome performance is parallelism.
  - Scaling vertically is limited to a finite amount of RAM, Threads and CPU speeds.
  - Scaling horizontally means we can simply add new "nodes" to the cluster almost endlessly.
- We parallelize at two levels:
  - The first level of parallelization is the **Executor** - a Java virtual machine running on a node, typically, one instance per node.
  - The second level of parallelization is the **Slot** - the number of which is determined by the number of cores and CPUs of each node.
- Each **Executor** has a number of **Slots** to which parallelized **Tasks** can be assigned to it by the **Driver**.

Spark API


Spark SQL + DataFrames
Streaming
MLlib
Spark Core API
R
SQL
Python
Scala
Java

Query Execution

We can express the same query using any interface. The Spark SQL engine generates the same query plan used to optimize and execute on our Spark cluster.



Resilient Distributed Datasets (RDDs) are the low-level representation of datasets processed by a Spark cluster. In early versions of Spark, you had to write code manipulating RDDs directly. In modern versions of Spark you should instead use the higher-level DataFrame APIs, which Spark automatically compiles into low-level RDD operation.

SparkSession

The **SparkSession** class is the single entry point to all functionality in Spark using the DataFrame API.

In Azure Synapse & Databricks notebooks, the SparkSession is created for you, stored in a variable called **spark**.

```
spark
```

```
from pyspark.sql import SparkSession

# Spark session & context
spark = SparkSession.builder.master("local").getOrCreate()
sc = spark.sparkContext

# Test PySpark
spark.range(5).show()
```

SparkSession Methods

Below are several additional methods we can use to create DataFrames. All of these can be found in the documentation for SparkSession.

Method	Description
sql	Returns a DataFrame representing the result of the given query
table	Returns the specified table as a DataFrame
read	Returns a DataFrameReader that can be used to read data in as a DataFrame
range	Create a DataFrame with a column containing elements in a range from start to end (exclusive) with step value and number of partitions
createDataFrame	Creates a DataFrame from a list of tuples, primarily used for testing

DataFrames

Queries using methods in the DataFrame API returns results in a DataFrame. A DataFrame is a distributed collection of data grouped into named columns.

```
budget_df = (spark
            .table("products")
            .select("name", "price")
            .where("price < 200")
            .orderBy("price")
           )
```

We can use display() to output the results of a dataframe.

```
display(budget_df)
```

Analyse data with Spark

```
%pyspark

# Load data
df=spark.read.load("/data/products.csv", format="csv", header=True)

# Manipulate dataframe
counts_df = df.select("ProductID", "Category").groupBy("Category").count()

# Display dataframe
display(counts_df)
```

Category	count
Headsets	3
Wheels	14
Mountain Bikes	32
…	…

Using SQL expressions in Spark

```
%pyspark

# Create a table in the metastore
df.createOrReplaceTempView("products")
```

```
%pyspark

# Use spark.sql method for inline SQL queries that return a dataframe
bikes_df = spark.sql("SELECT ProductID, ProductName, ListPrice \
FROM products \
WHERE Category IN ('Mountain Bikes', 'Road Bikes')")

display(bikes_df)
```

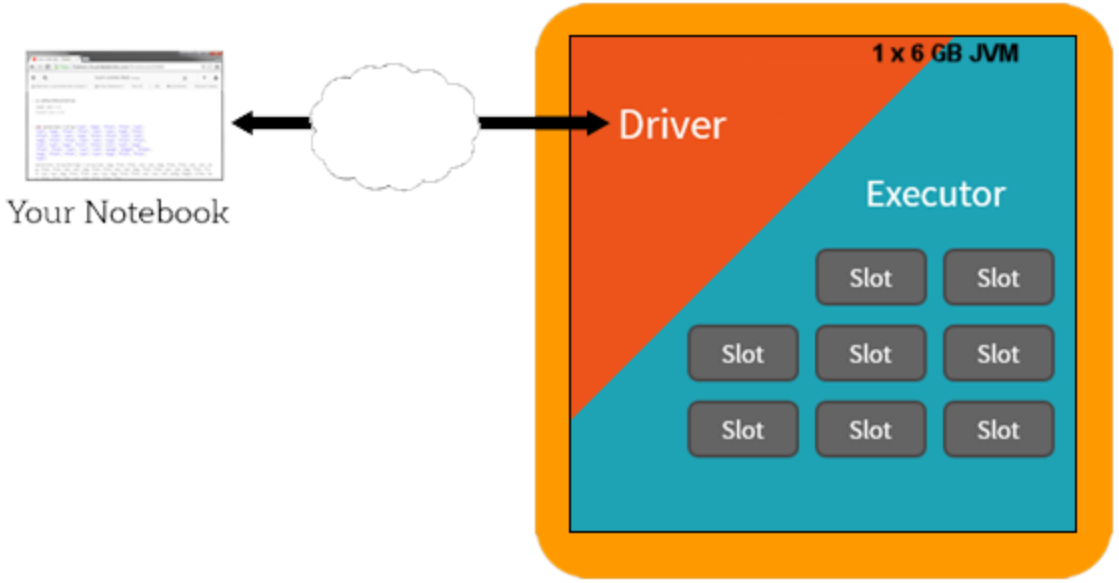
```
%sql

-- Use SQL to query tables in the metastore
SELECT Category, COUNT(ProductID) AS ProductCount
FROM products
GROUP BY Category
ORDER BY Category
```

Spark Local Mode

One option, commonly used for local development, is to run Spark in **Local Mode**. In **Local Mode**, both the **Driver** and one **Executor** share the same JVM.

- This is an ideal scenario for experimentation, prototyping, and learning!



Running PySpark on Jupyter Notebook with Docker on Mac: A Step-by-Step Guide



This article aims to simplify the process by demonstrating how to run PySpark seamlessly on Jupyter Notebook using Docker, specifically tailored for Mac users. Docker, with its containerization magic, ensures a consistent and reproducible environment, while Jupyter Notebook provides an interactive and user-friendly interface for data exploration and analysis.

```
# Import necessary libraries
from pyspark.sql import SparkSession

# Create a Spark session
spark = SparkSession.builder.appName("pyspark-jupyter").getOrCreate()

# Define the path to the data file within the "sparkdata" volume
data_path = "/sparkdata/taxi_trip/taxi_zone_lookup.csv"

# Load the data into a PySpark DataFrame
df = spark.read.csv(data_path, header=True, inferSchema=True)

# Show the first few rows of the DataFrame
df.show()
```

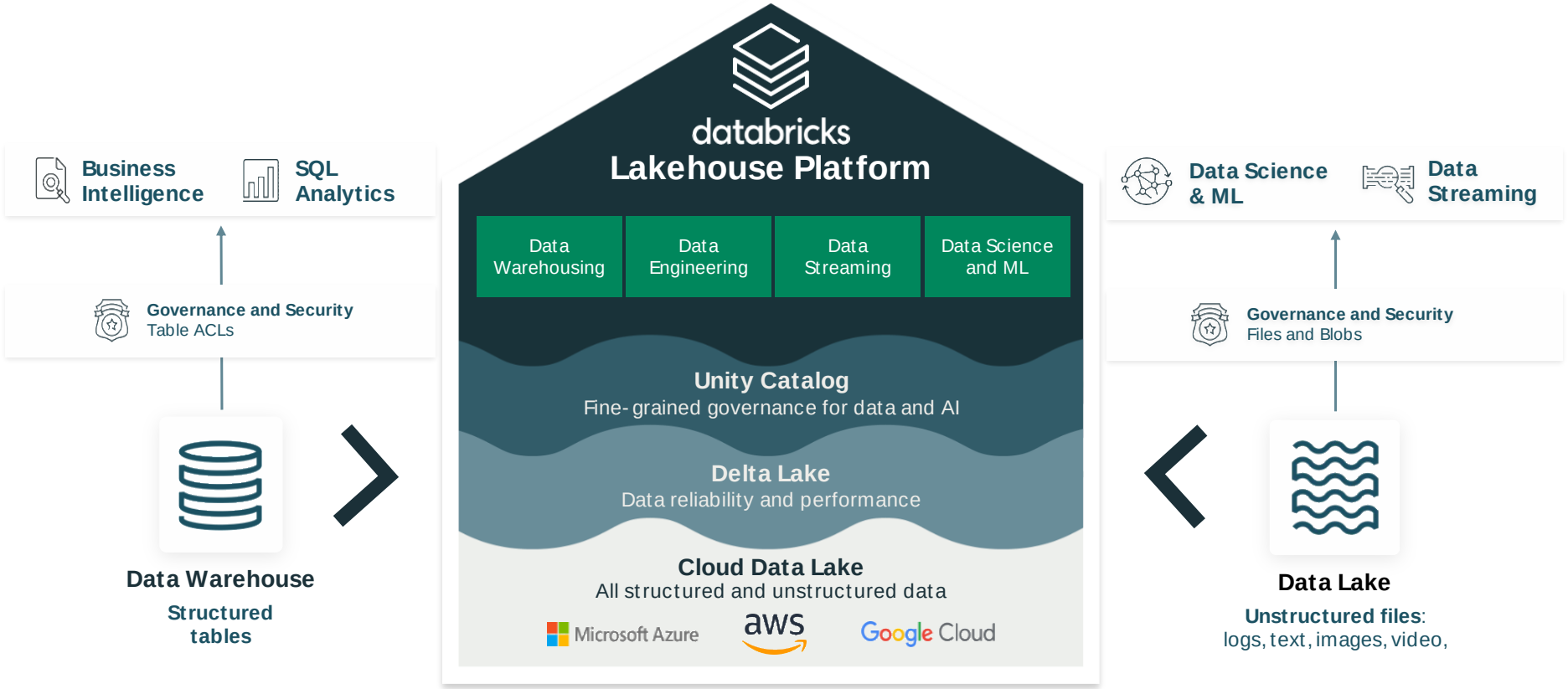
https://medium.com/p/0b2e3bad1930

Understand the core components of the Databricks Lakehouse platform

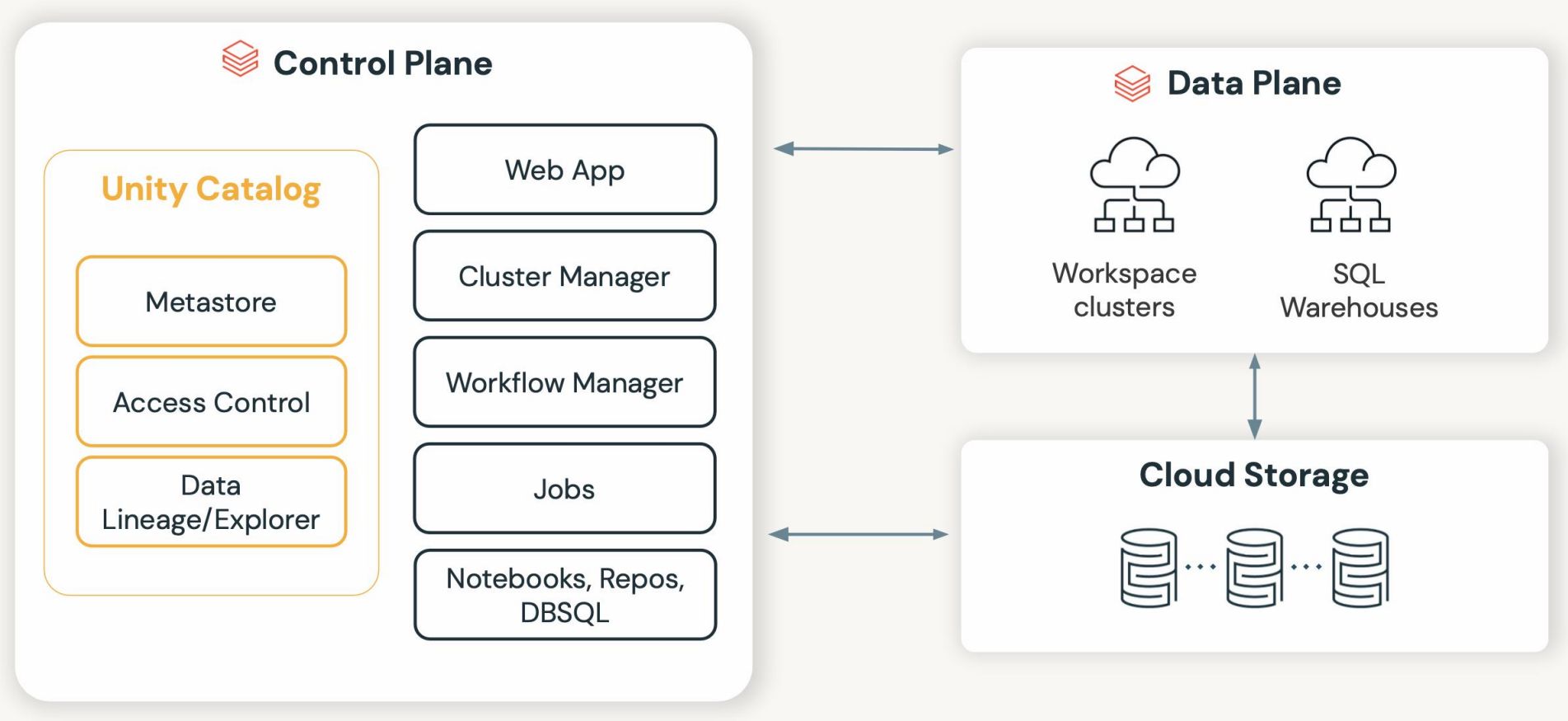


What is Databricks?

One platform to unify all your data, analytics and AI workloads.



Databricks Workspace and Services



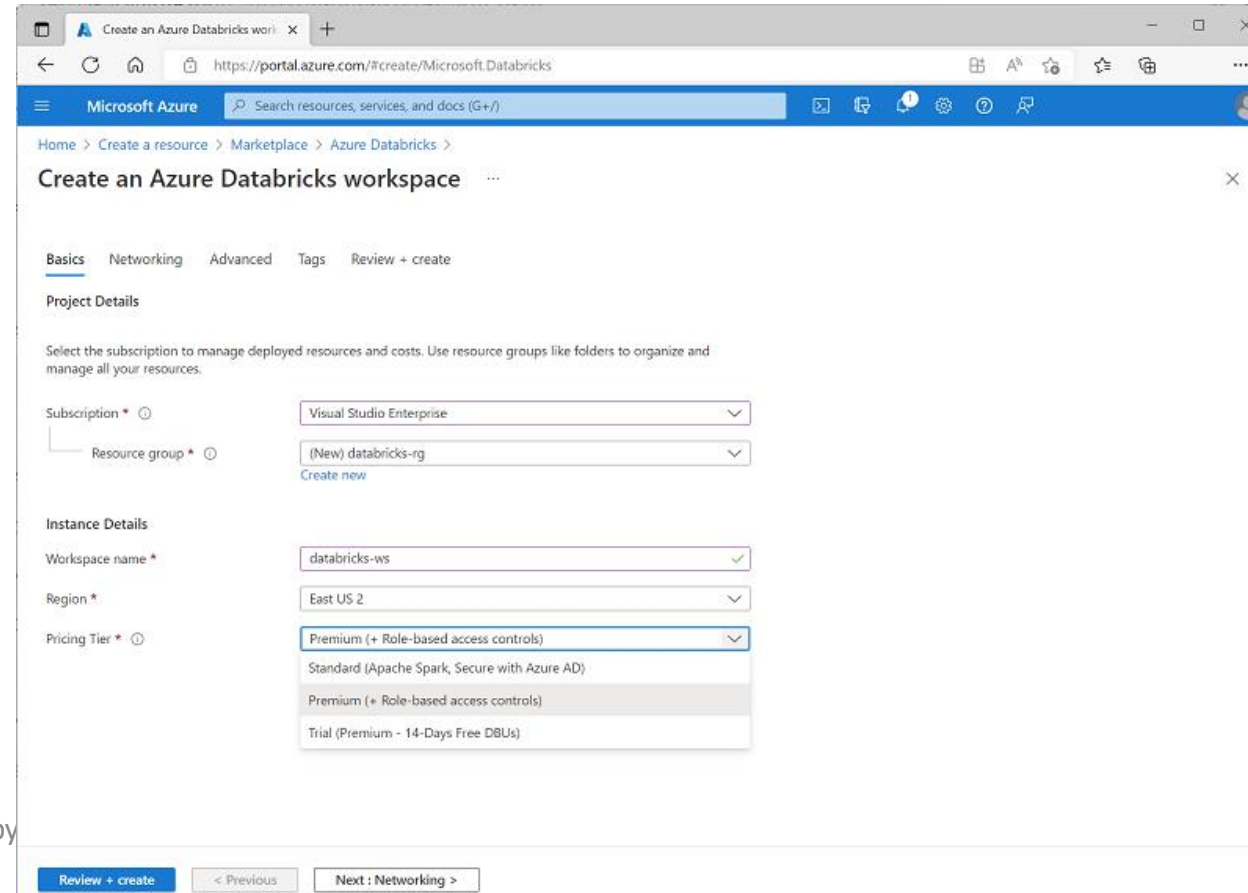
What is Azure Databricks?

Databricks is a fully managed, cloud-based data analytics platform

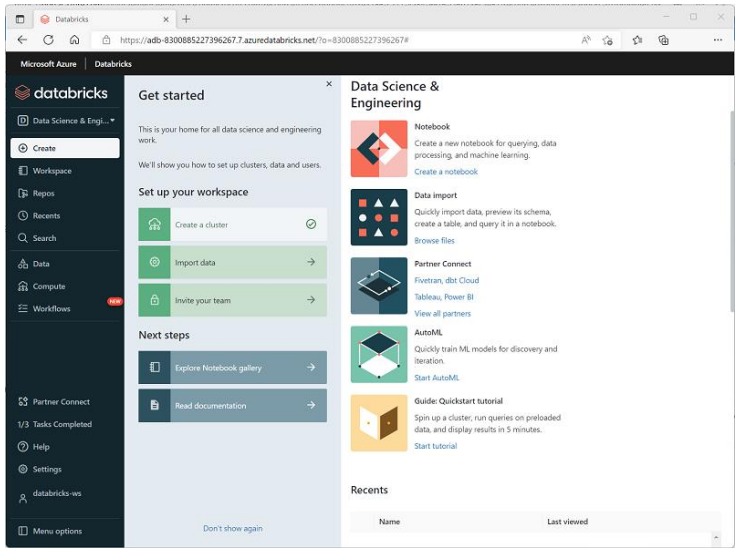
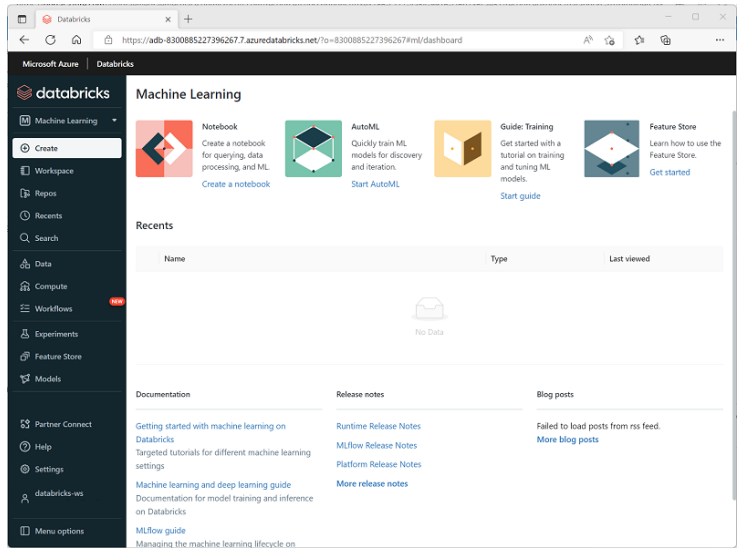
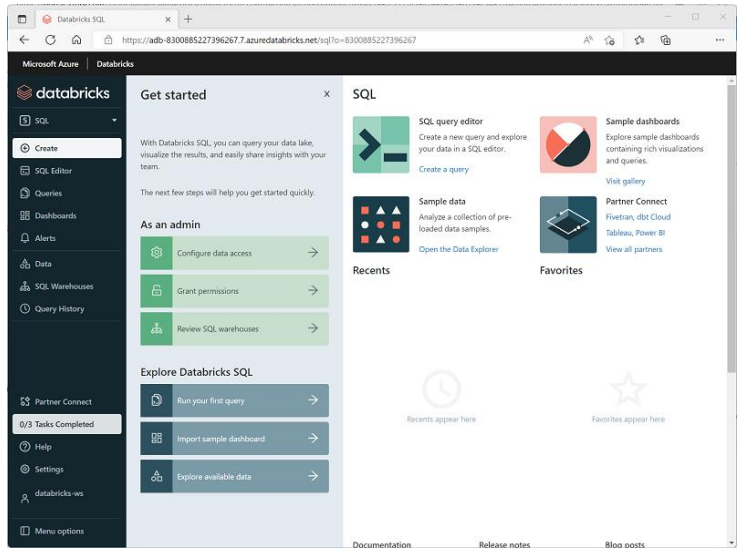
- Built on Apache Spark
- Web-based portal

Provisioned as an Azure resource

- Standard tier
- Premium tier
- Trial

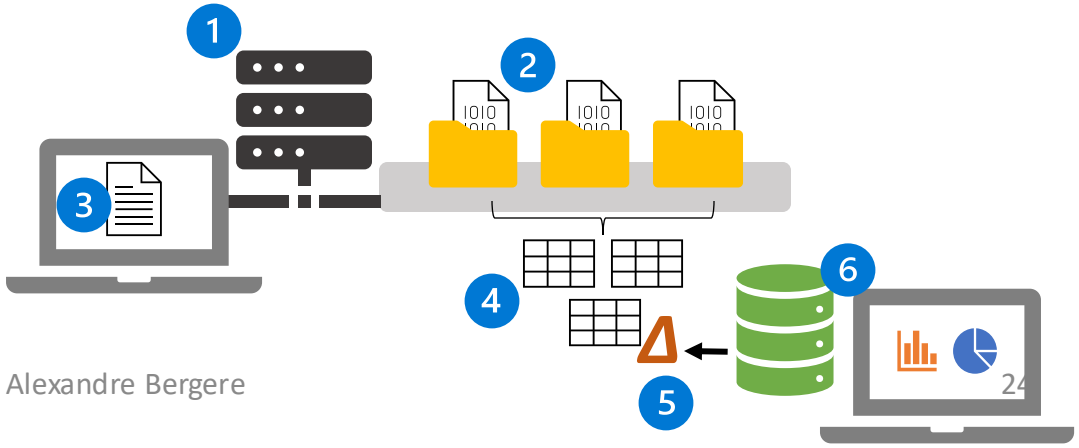


Azure Databricks workload

Data Science and Engineering	Machine Learning	SQL
		
Use notebooks to run Apache Spark code to manipulate and explore data	Train predictive models using SparkML and other machine learning frameworks	Store and query data in relational tables using SQL Only available in *Premium* tier workspaces

Key concepts

1. **Apache Spark clusters** provide highly scalable parallel compute for distributed data processing
2. **Databricks File System (DBFS)** provides distributed shared storage for data lakes
3. **Notebooks** provide an interactive environment for combining code, notes, and images
4. **Metastore** provides a relational abstraction layer, enabling you to define tables based on data in files
5. **Delta Lake** builds on the metastore to enable common relational database capabilities
6. **SQL Warehouses** provide relational compute endpoints for querying data in tables



Databricks File System

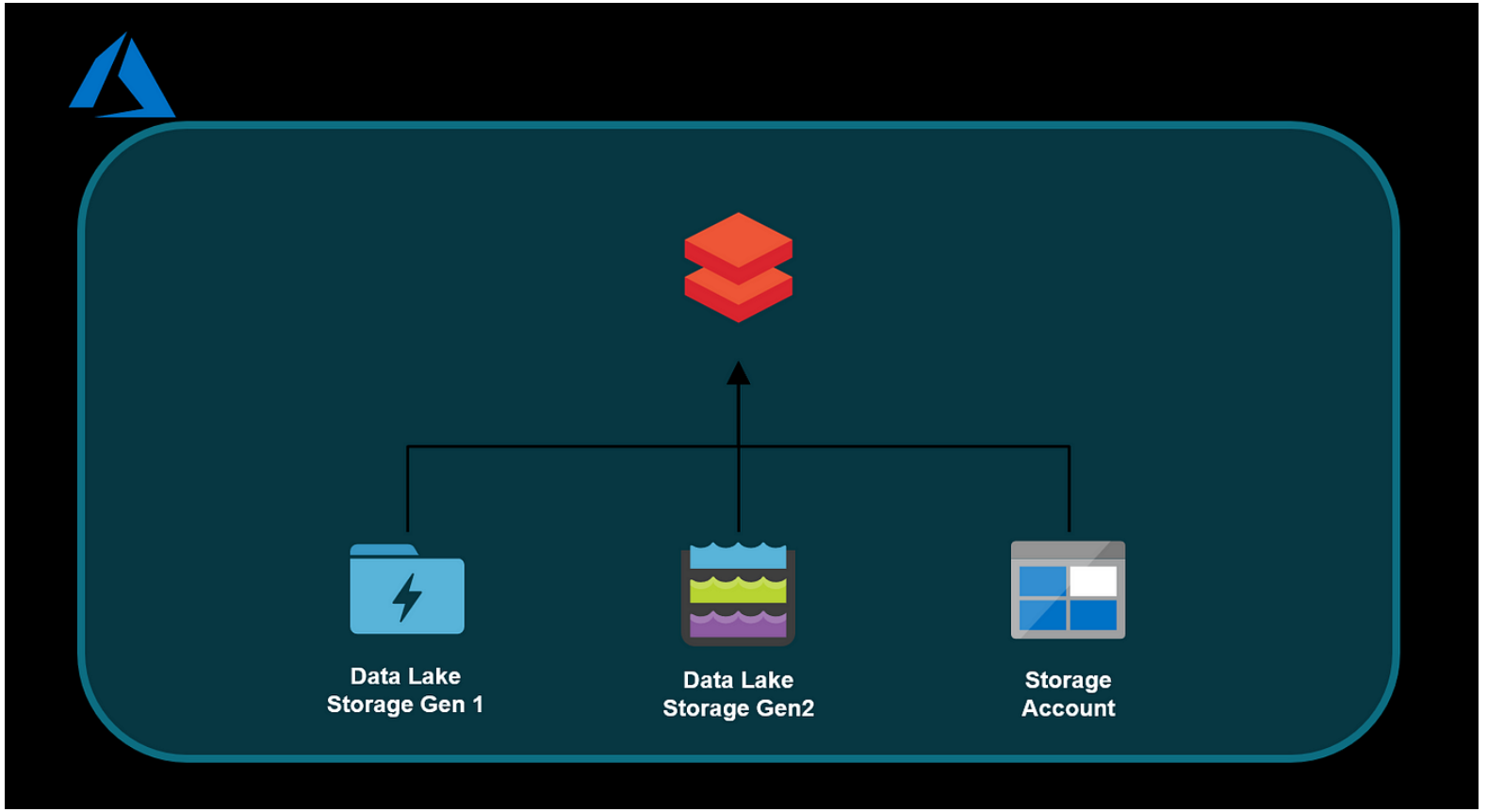
Databricks File System (DBFS) is a distributed file system mounted into a Databricks workspace and available on Databricks clusters. DBFS is an abstraction on top of scalable object storage and offers the following benefits:

- Allows you to mount storage objects so that you can seamlessly access data without requiring credentials.
- Allows you to interact with object storage using directory and file semantics instead of storage URLs.
- Files persist to object storage, so you won't lose data after you terminate a cluster.

https://docs.databricks.com/user-guide/databricks-file-system.html#dbfs

Azure Databricks — Connect Azure storage to Databricks





https://medium.com/datalex/azure-databricks-connect-azure-storage-to-databricks-285cac910fa0

Reprise à 16h05

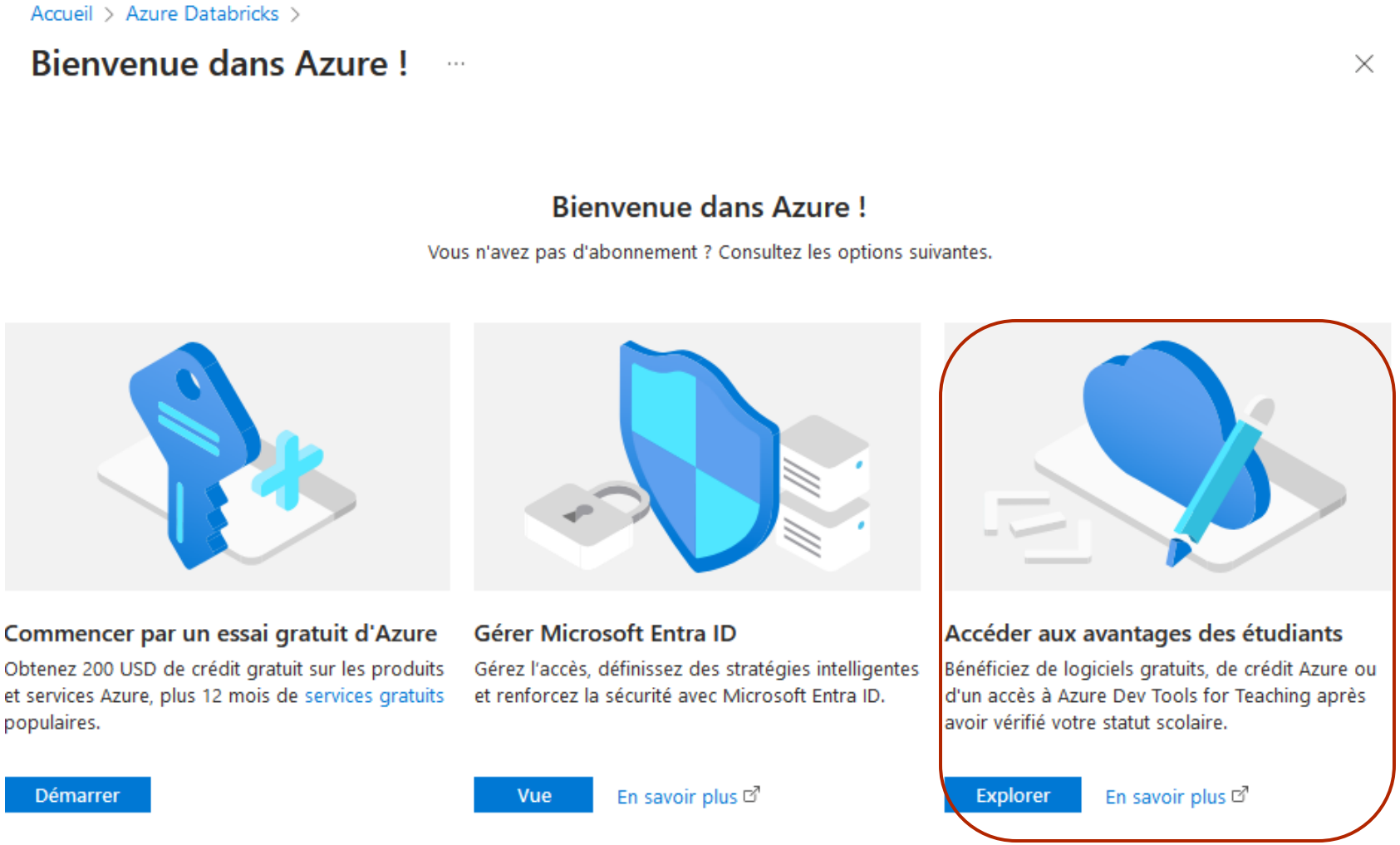


datalex

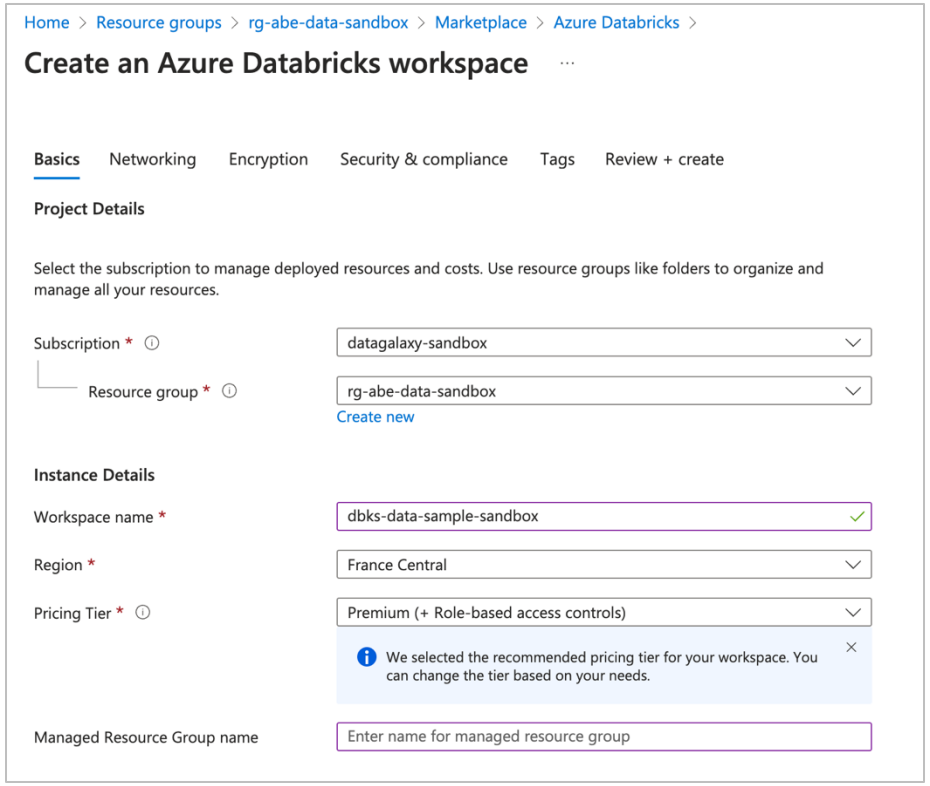
Demo - Navigate the Workspace UI

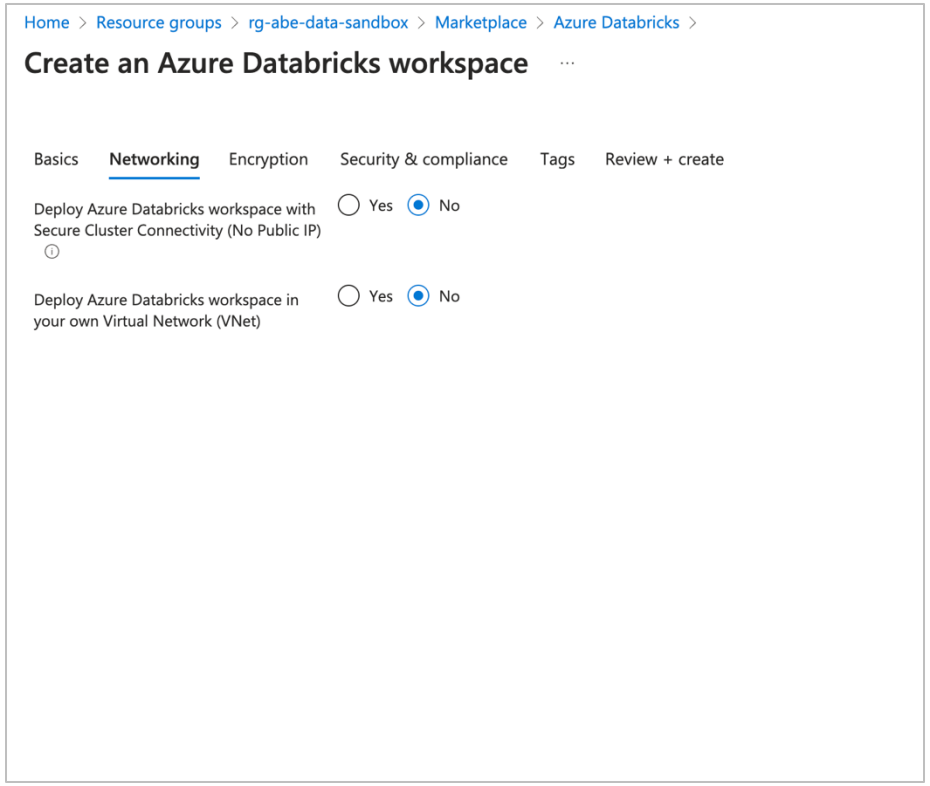


Enable student credits

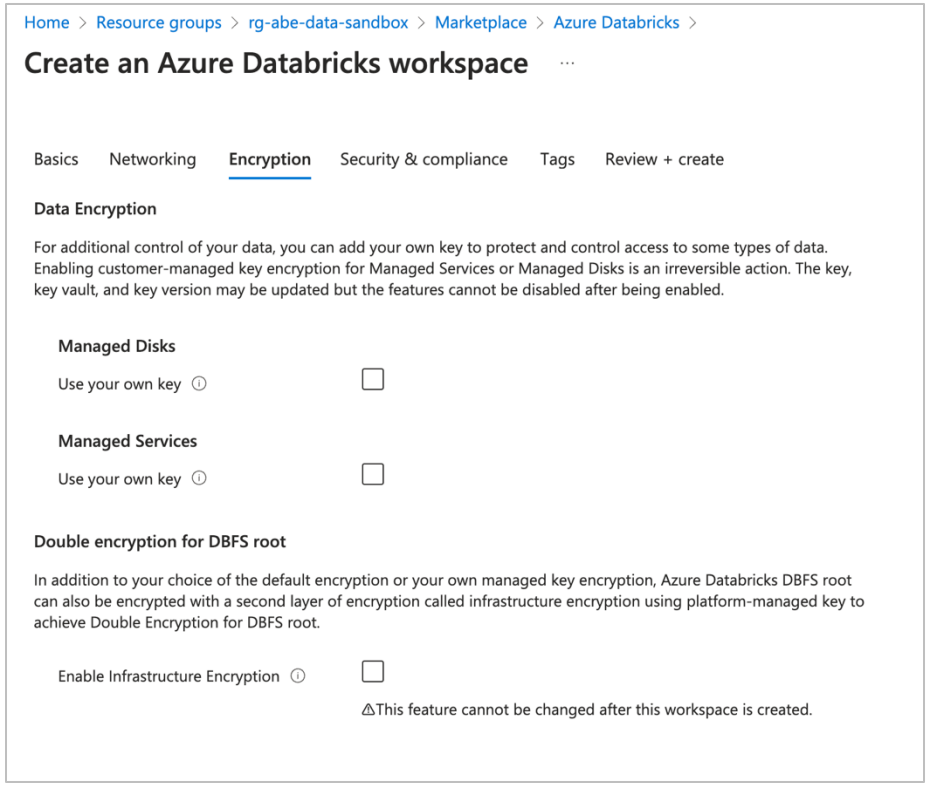


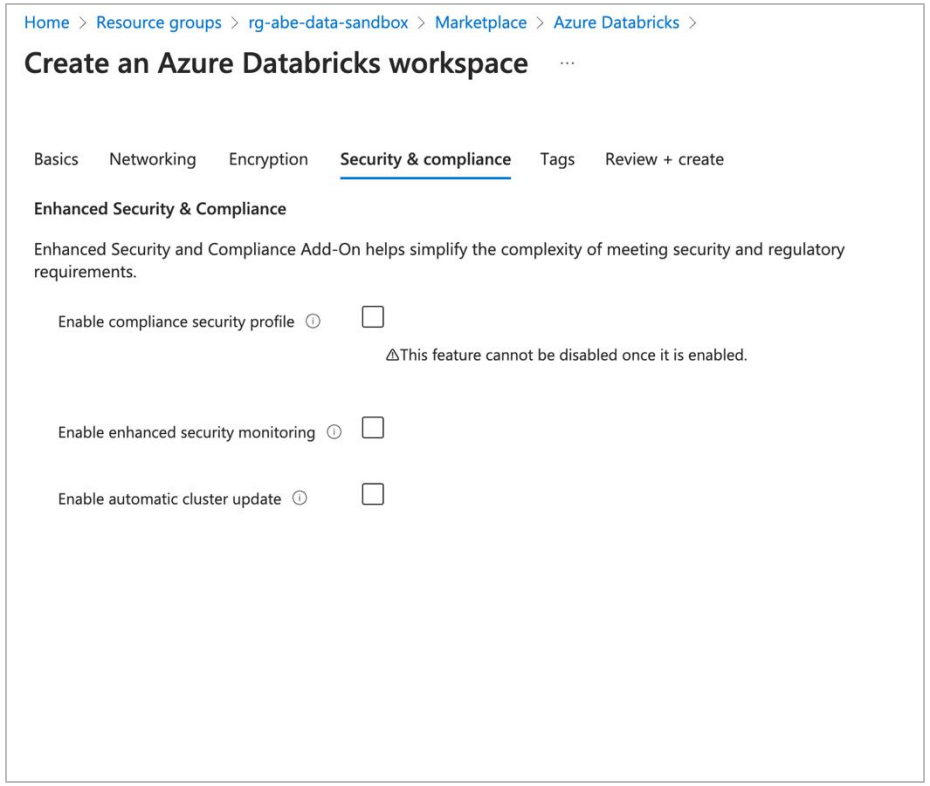
Create an Azure Databricks Workspace



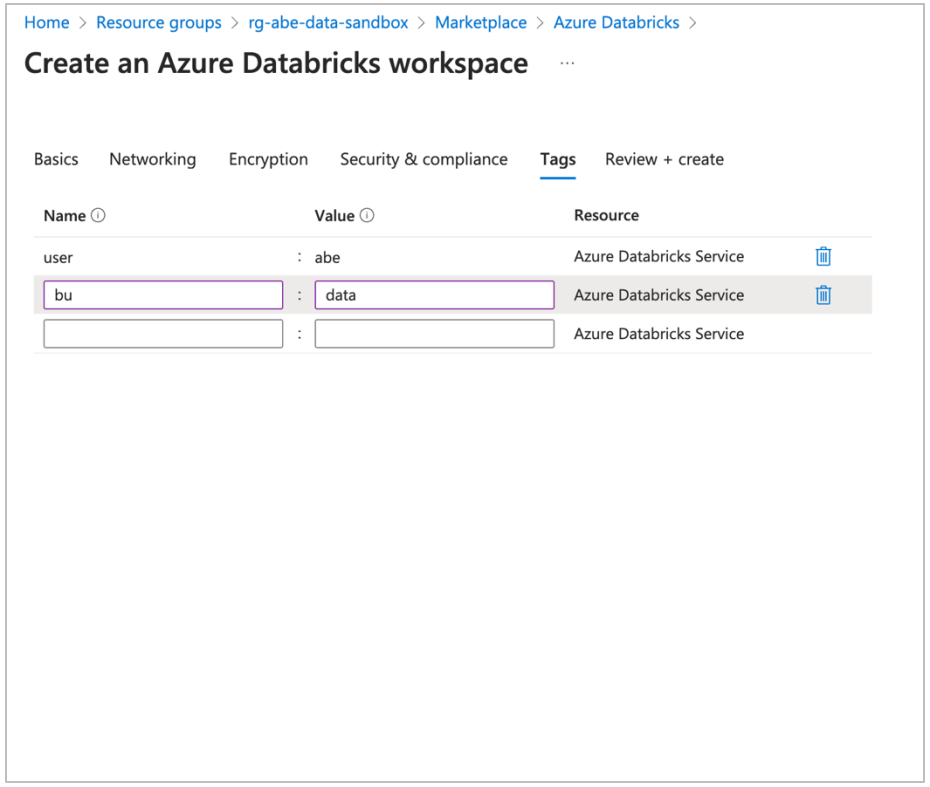


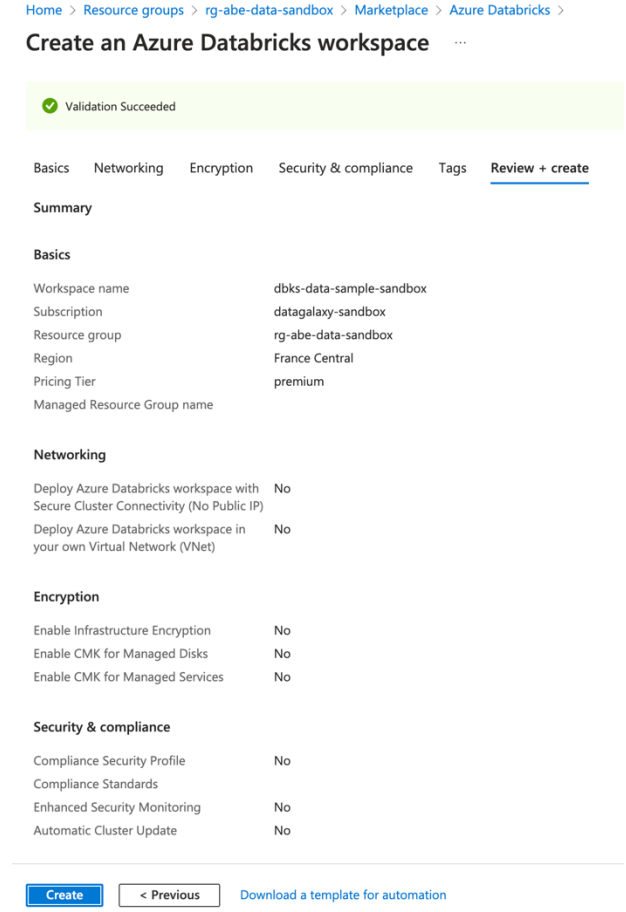
Create an Azure Databricks Workspace



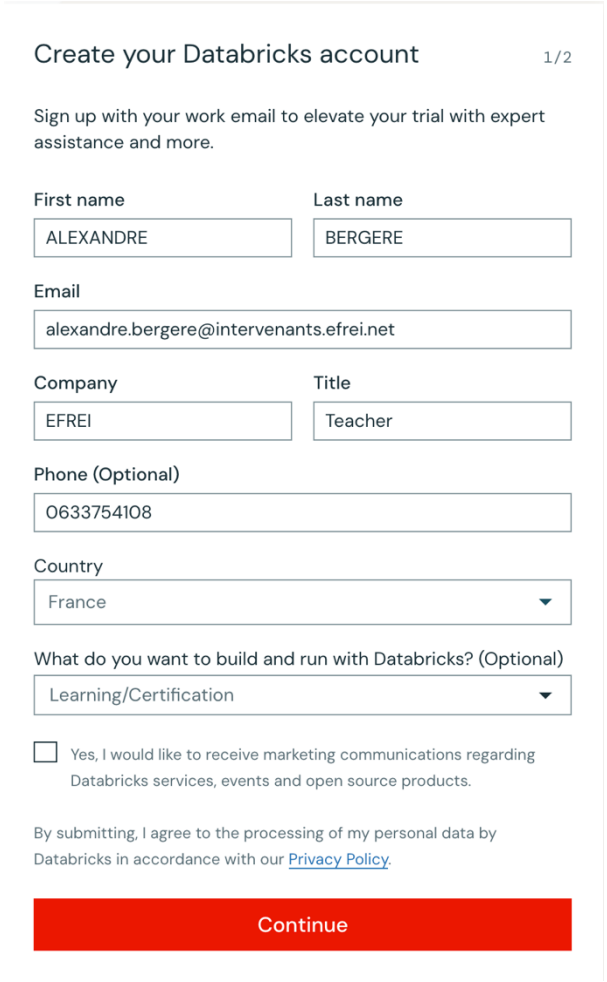


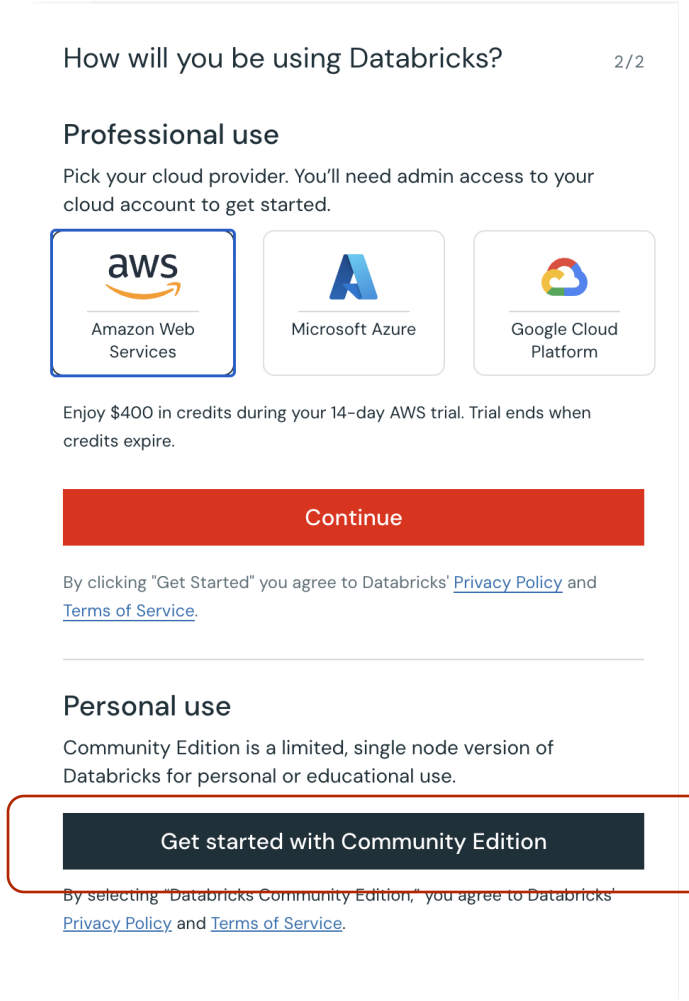
Create an Azure Databricks Workspace





Use Databricks Community Edition





Compute Ressources



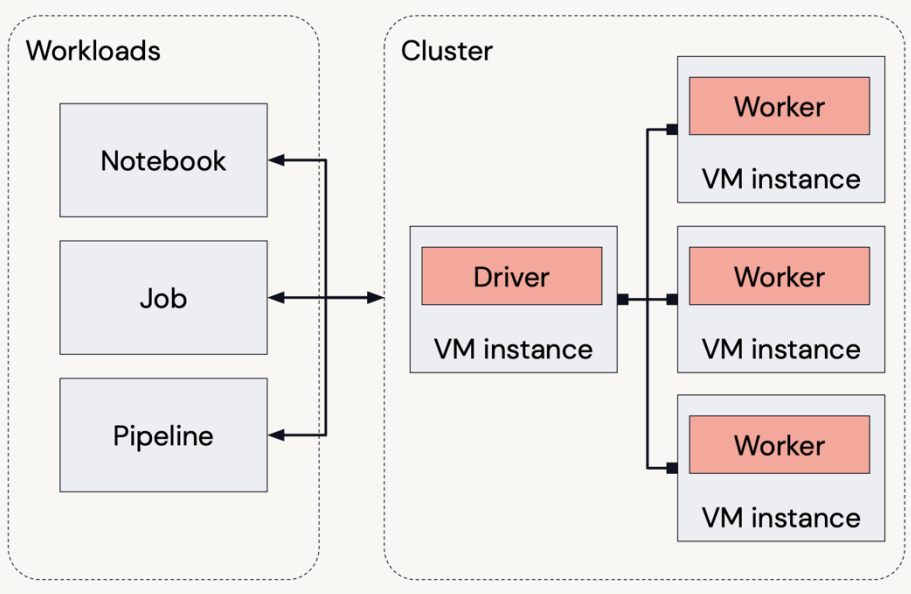
Clusters

Collection of VM instances

Distributes workloads across workers

Two main types:

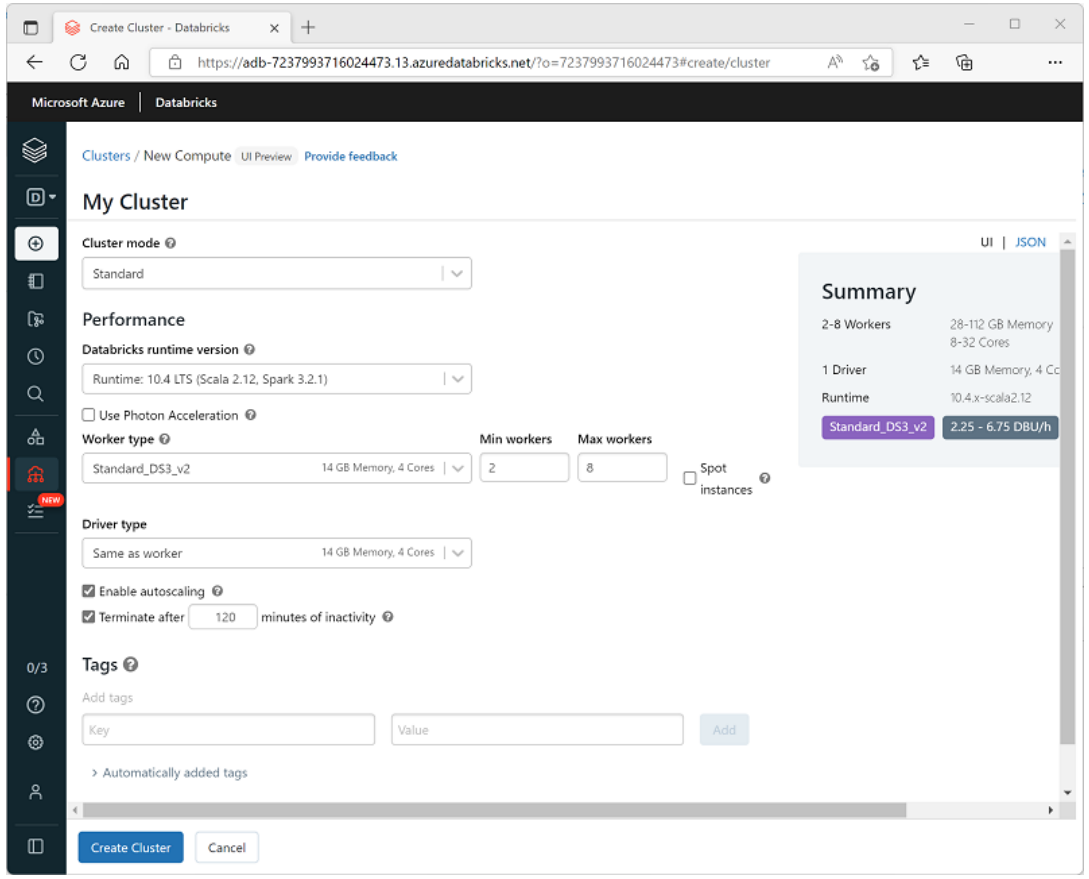
- **All-purpose** clusters for interactive development
- **Job** clusters for automating workloads



Create a Spark cluster

Create a cluster in the Azure Databricks portal, specifying:

- Cluster name
- Cluster mode (standard, high-concurrency, or single-node)
- Databricks Runtime version
- Worker and driver node VM configuration
- Autoscaling and automatic shutdown



Cluster Types

All-purpose Clusters

- Analyze data collaboratively using **interactive** notebooks
- Create clusters from the Workspace or API
- Configuration information retained for up to 70 clusters for up to 30 days

Job Clusters Run **automated** jobs

- The Databricks job scheduler creates job clusters when running jobs
- Configuration information retained for up to 30 most recently terminated clusters

Cluster Configuration

Cluster Mode

Standard (Multi Node)
Default mode for workloads developed in any supported language (requires at least two VM instances)

Single node
Low-cost single-instance cluster catering to single-node machine learning workloads and lightweight exploratory analysis

Cluster Configuration

Databricks Runtime Version

Standard
Apache Spark and many other components and updates to provide an optimized big data analytics experiences

Machine learning
Adds popular machine learning libraries like TensorFlow, Keras, PyTorch, and XGBoost.

Photon
An optional add-on to optimize SQL workloads

Cluster Configuration

Access Mode

Access mode dropdown	Visible to user	Unity Catalog support	Supported languages
Single user	Always	Yes	Python, SQL, Scala, R
Shared	Always (Premium plan required)	Yes	Python (DBR 11.1+), SQL
No isolation shared	Can be hidden by enforcing user isolation in the admin console or configuring account-level settings	No	Python, SQL, Scala, R
Custom	Only shown for existing clusters *without* access modes (i.e. legacy cluster modes, Standard or High Concurrency); not an option for creating new clusters.	No	Python, SQL, Scala, R

Cluster Configuration

Cluster Policies

Cluster policies can help to achieve the following:

- Standardize cluster configurations
- Provide predefined configurations targeting specific use cases
- Simplify the user experience
- Prevent excessive use and control cost
- Enforce correct tagging

Cluster Configuration

Cluster Access Control

	No Permissions	Can Attach To	Can Restart	Can Manage
Attach notebook		✔	✔	✔
View Spark UI, cluster metrics, driver logs		✔	✔	✔
Start, restart, terminate			✔	✔
Edit				✔
Attach library				✔
Resize				✔
Change permissions				✔

DE 1.1: Create and Manage Interactive Clusters

- Use the Clusters UI to configure and deploy a cluster
- Edit, terminate, restart, and delete clusters



Develop Code with Notebooks



Databricks Notebooks

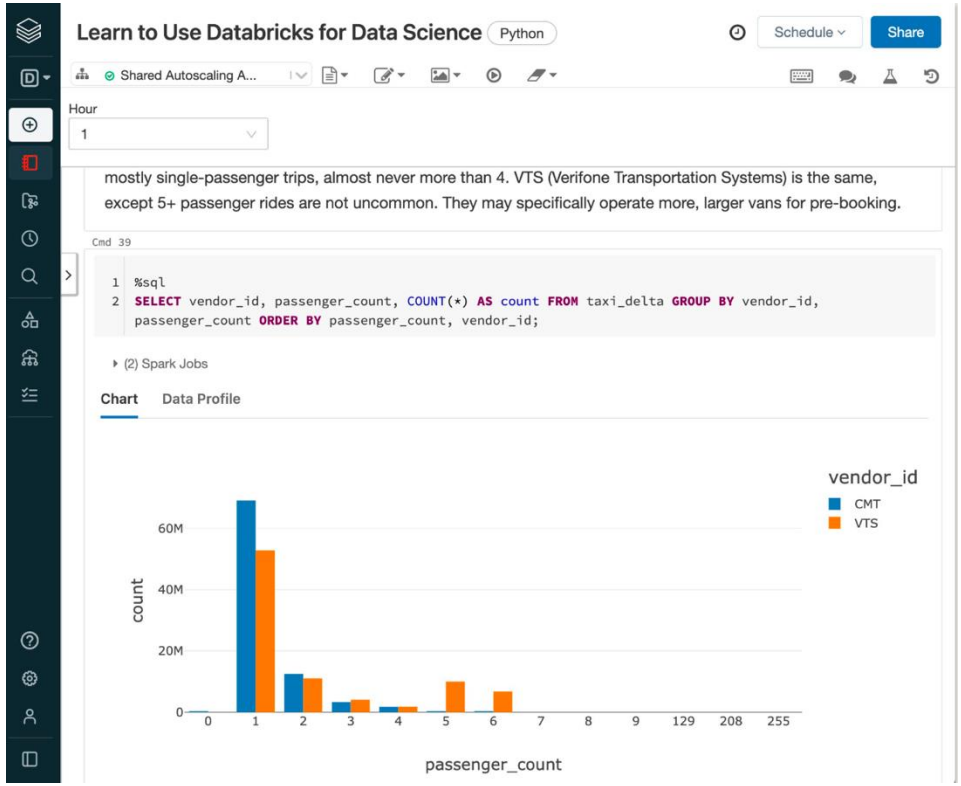
Collaborative, reproducible, and enterprise ready

Multi-language
Use Python, SQL, Scala, and R, all in one Notebook

Collaborative
Real-time co-presence, co-editing, and commenting

Ideal for exploration
Explore, visualize, and summarize data with built-in charts and data profiles

Adaptable
Install standard libraries and use local modules



Reproducible
Automatically track version history, and use git version control with Repos

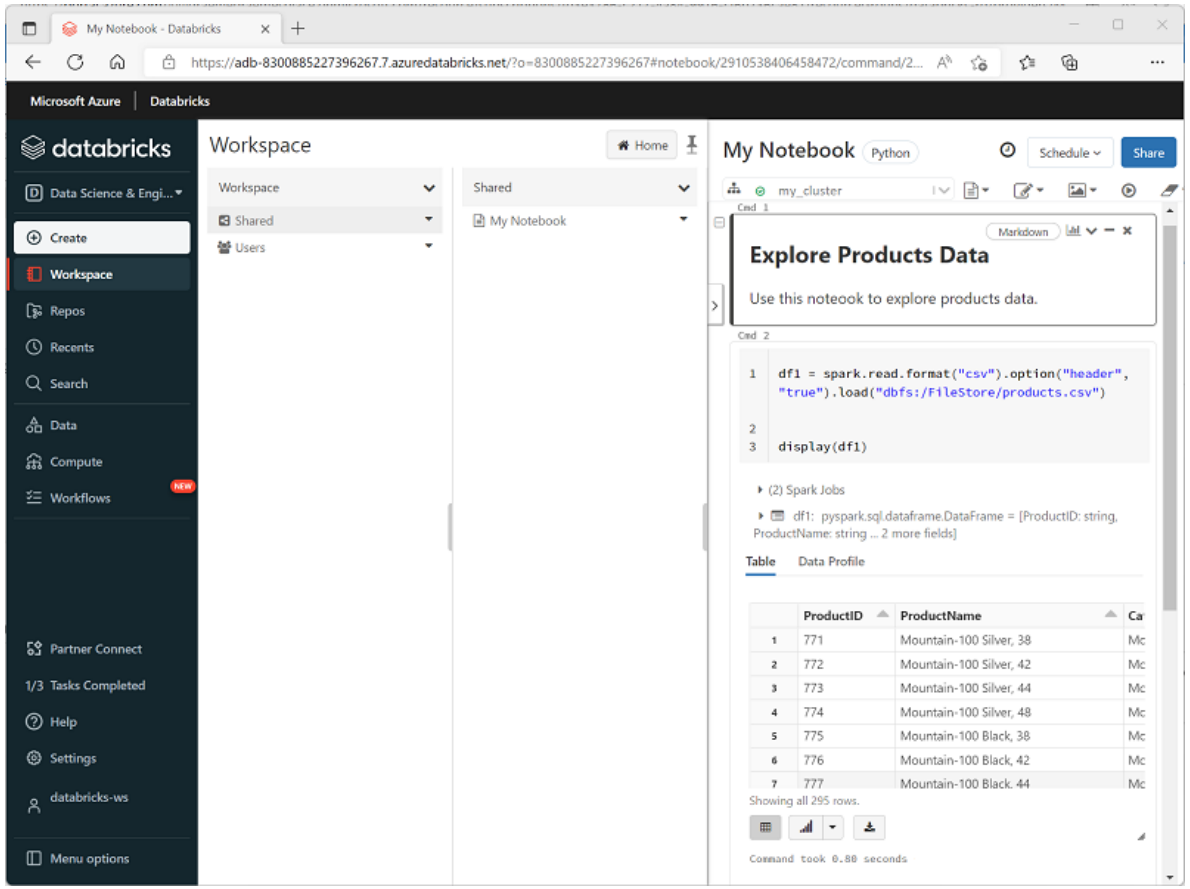
Get to production faster
Quickly schedule notebooks as jobs or create dashboards from their results, all in the Notebook

Enterprise-ready
Enterprise-grade access controls, identity management, and auditability

Use Spark in notebooks

Interactive notebooks

- Syntax highlighting and error support
- Code auto-completion
- Interactive data visualizations
- The ability to export results



Notebook magic commands

Use to override default languages, run utilities/auxiliary commands, etc.

- %python, %r, %scala, %sql — Switch languages in a command cell
- %sh — Run shell code (only runs on driver node, not worker nodes)
- %fs — Shortcut for dbutils filesystem commands
- %md — Markdown for styling the display
- %run — Execute a remote notebook from a notebook
- %pip — Install new Python libraries

dbutils (Databricks Utilities)

Perform various tasks with Databricks using notebooks.

Utility	Description	Example
fs	Manipulates the Databricks filesystem (DBFS) from the console	dbutils.fs.ls()
secrets	Provides utilities for leveraging secrets within notebooks	dbutils.secrets.get()
notebook	Utilities for the control flow of a notebook	dbutils.notebook.run()
widgets	Methods to create and get bound value of input widgets inside notebooks	dbutils.widget.text()
jobs	Utilities for leveraging jobs features	dbutils.jobs.taskValues.set()

Available within Python, R, or Scala notebooks.

Git Versioning with Databricks Repos

Databricks Repos

Git Versioning

Native integration with Github, Gitlab, Bitbucket and Azure Devops

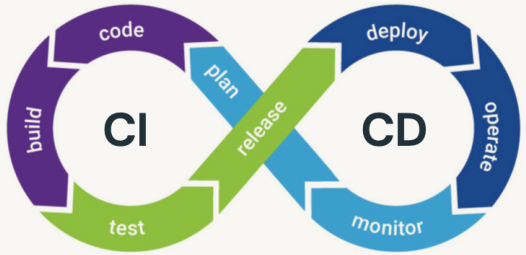
UI-based workflows

Azure DevOps GitHub GitLab Bitbucket

CI/CD Integration

API surface to integrate with automation

Simplifies the dev/staging/prod multi-workspace story



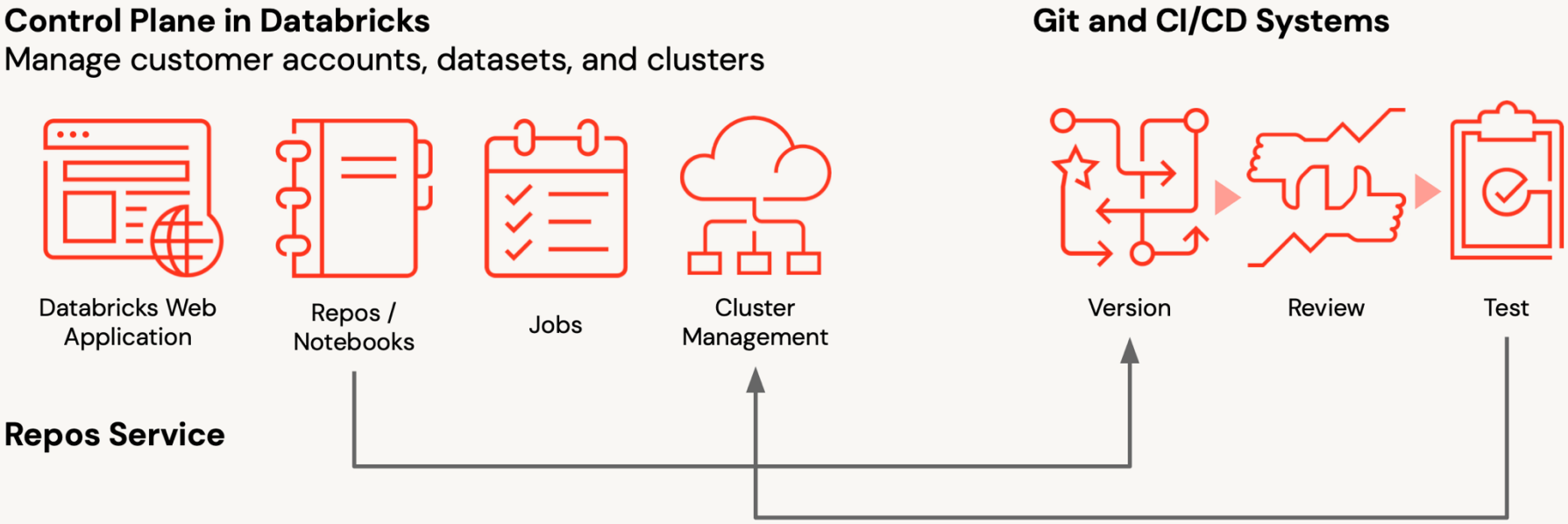
Enterprise ready

Allow lists to avoid exfiltration

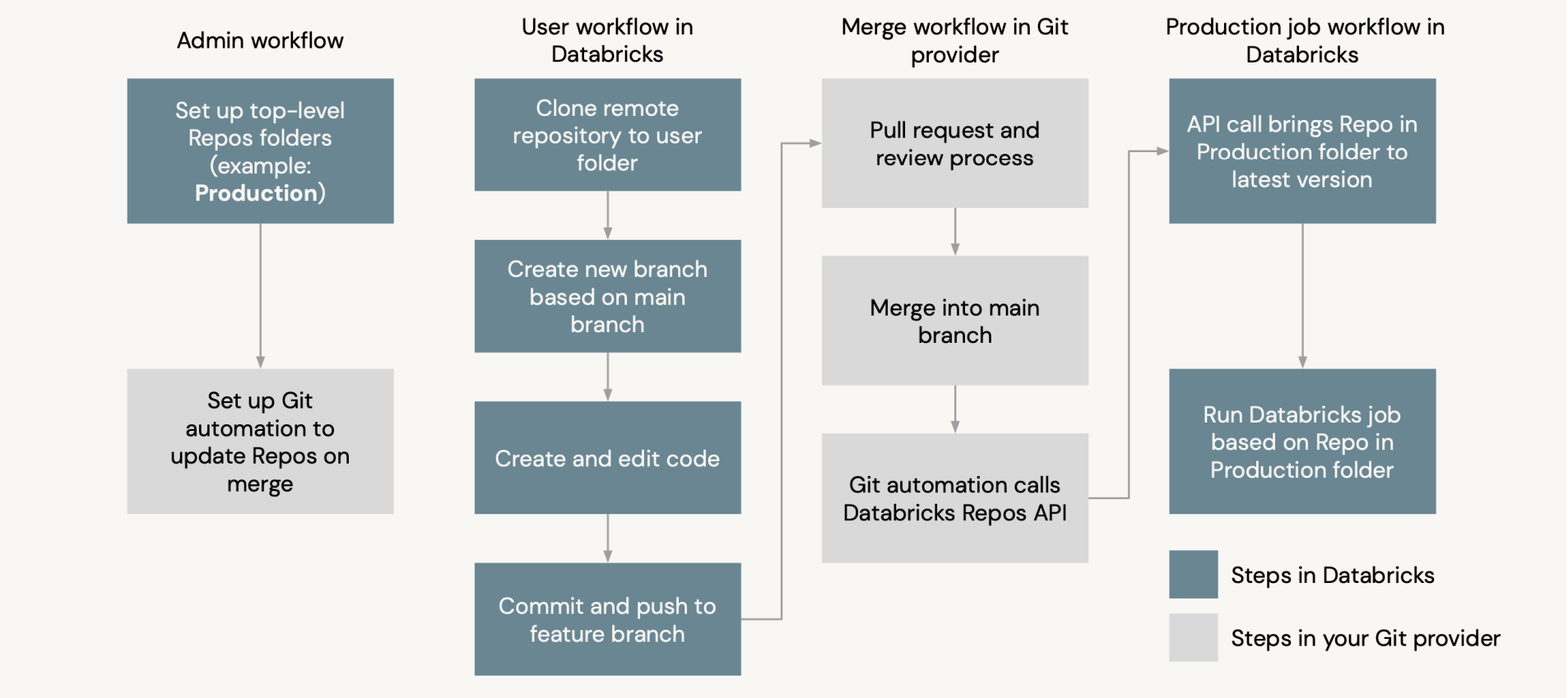
Secret detection to avoid leaking keys

Databricks Repos

CI/CD Integration



CI/CD workflows with Git and Repos



https://www.google.com/url?q=https://docs.databricks.com/repos/ci-cd-techniques-with-repos.html&sa=D&source=editors&ust=1683908557263917&usg=AOvVaw2XjB4PcscoUZbtANSbp4pl

DE 1.2: Databricks Notebook Operations

- Attach a notebook to a cluster to execute a cell in a notebook Set the default language for a notebook
- Describe and use magic commands
- Create and run SQL, Python, and markdown cells
- Export a single or collection of notebook

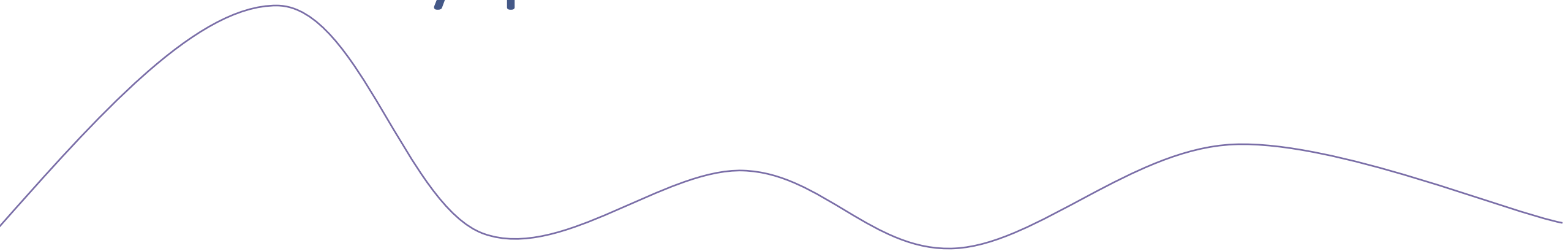


DE 1.3L: Get Started with the Databricks Platform



- Rename a notebook and change the default language Attach a cluster
- Use the %run magic command
- Run Python and SQL cells
- Create a Markdown cell

Use PySpark in Azure Databricks



DataFrameReader

Interface used to load a DataFrame from external storage systems

```
spark.read.csv("path/to/files").option("header", "true")
```

```
spark.read.format("csv").option("header", "true").load("path/to/files")
```

DataFrameReader is accessible through the SparkSession attribute read. This class includes methods to load DataFrames from different external storage systems.

Quick function review:

- csv(path)
- jdbc(url, table, ..., connectionProperties)
- json(path)
- format(source)
- load(path)
- orc(path)
- parquet(path)
- table(tableName)
- text(path)
- textFile(path)

Configuration methods:

- option(key, value)
- options(map)
- schema(schema)

File format

A file format is a standard way that information is encoded for storage in a computer file. File formats may be either proprietary or free and may be either unpublished or open.

Categorie	File Formats
Images	PNG, MNG, TIFF, JPEG, GIF, TGA, OpenEXR, BMP, FITS
Graphic	VML, SVG, Silverlight, SWF, AI, EPS, DXF
Audio	OGG, FLAC, MP3, WAV, WMA, AAC
Video	MPEG, OGM (DVD, DivX, XviD), AVI, Theora, FLV, MP4
Page	PDF, PostScript, HTML, XHTML, XML
Document	JSON, BSON
Word processing document	ODT, TXT, DOC, RTF
Executable	BIN, ELF, EXE, SDC, BAT
Archive	7Z, TAR, GZIP, ZIP, LZW, ARJ, RAR, SDC
Plain text storage	CSV, TSV, TXT
Columnar Big Data File Formats	Parquet, Avro, ORC
Columnar Big Data Table Formats	Delta Lake, Iceberg, Huidi

Plain text storage

Code INSEE	Commune	Département	Région
32460	VERGOIGNAN	GERS	MIDI-PYRENEES
51141	LA CHAUSSEE-SUR-MARNE	MARNE	CHAMPAGNE-ARDENNE
77130	COULOMMES	SEINE-ET-MARNE	ILE-DE-FRANCE
63379	SAINT-MYON	PUY-DE-DOME	AUVERGNE
62050	AUCHY-LES-HESDIN	PAS-DE-CALAIS	NORD-PAS-DE-CALAIS
13020	CADOLIVE	BOUCHES-DU-RHONE	PROVENCE-ALPES-COTE D'AZUR
94017	CHAMPIGNY-SUR-MARNE	VAL-DE-MARNE	ILE-DE-FRANCE
95370	MARINES	VAL-D'OISE	ILE-DE-FRANCE
87062	EYBOULEUF	HAUTE-VIENNE	LIMOUSIN
37159	MONTS	INDRE-ET-LOIRE	CENTRE
38286	OULLES	ISERE	RHONE-ALPES
46253	SAINT-CHAMARAND	LOT	MIDI-PYRENEES
80579	MUILLE-VILLETTE	SOMME	PICARDIE
52204	FORCEY	HAUTE-MARNE	CHAMPAGNE-ARDENNE
60187	CUIGY-EN-BRAY	OISE	PICARDIE
24161	EGLISE-NEUVE-D'ISSAC	DORDOGNE	AQUITAINE
6149	LA TRINITE	ALPES-MARITIMES	PROVENCE-ALPES-COTE D'AZUR
2A141	LETIA	CORSE-DU-SUD	CORSE
30036	BERNIS	GARD	LANGUEDOC-ROUSSILLON
72019	AVESSE	SARTHE	PAYS DE LA LOIRE
72087	CONFLANS-SUR-ANILLE	SARTHE	PAYS DE LA LOIRE
16347	SAINT-ROMAIN	CHARENTE	POITOU-CHARENTES
58108	EMPURY	NIEVRE	BOURGOGNE
54387	MORVILLE-SUR-SEILLE	MEURTHE-ET-MOSELLE	LORRAINE
22190	PLESLIN-TRIGAVOU	COTES-D'ARMOR	BRETAGNE
62604	NEUFCHATEL-HARDELOT	PAS-DE-CALAIS	NORD-PAS-DE-CALAIS
53039	LE BOURGNEUF-LA-FORET	MAYENNE	PAYS DE LA LOIRE
55397	PAGNY-LA-BLANCHE-COTE	MEUSE	LORRAINE
60645	TRIE-LA-VILLE	OISE	PICARDIE
39505	SAUGEOT	JURA	FRANCHE-COMTE
88100	CHEF-HAUT	VOSGES	LORRAINE
44131	PORNIC	LOIRE-ATLANTIQUE	PAYS DE LA LOIRE
59249	FOURMIES	NORD	NORD-PAS-DE-CALAIS
17005	ALLAS-BOCAGE	CHARENTE-MARITIME	POITOU-CHARENTES
2668	SAINS-RICHAUMONT	AISNE	PICARDIE




```
Code INSEE;Commune;Département;Région
32460;VERGOIGNAN;GERS;MIDI-PYRENEES
51141;LA CHAUSSEE-SUR-MARNE;MARNE;CHAMPAGNE-ARDENNE
77130;COULOMMES;SEINE-ET-MARNE;ILE-DE-FRANCE
63379;SAINT-MYON;PUY-DE-DOME;AUVERGNE
62050;AUCHY-LES-HESDIN;PAS-DE-CALAIS;NORD-PAS-DE-CALAIS
13020;CADOLIVE;BOUCHES-DU-RHONE;PROVENCE-ALPES-COTE D'AZUR
94017;CHAMPIGNY-SUR-MARNE;VAL-DE-MARNE;ILE-DE-FRANCE
95370;MARINES;VAL-D'OISE;ILE-DE-FRANCE
87062;EYBOULEUF;HAUTE-VIENNE;LIMOUSIN
37159;MONTS;INDRE-ET-LOIRE;CENTRE
38286;OULLES;ISERE;RHONE-ALPES
46253;SAINT-CHAMARAND;LOT;MIDI-PYRENEES
80579;MUILLE-VILLETTE;SOMME;PICARDIE
52204;FORCEY;HAUTE-MARNE;CHAMPAGNE-ARDENNE
60187;CUIGY-EN-BRAY;OISE;PICARDIE
24161;EGLISE-NEUVE-D'ISSAC;DORDOGNE;AQUITAINE
6149;LA TRINITE;ALPES-MARITIMES;PROVENCE-ALPES-COTE D'AZUR
2A141;LETIA;CORSE-DU-SUD;CORSE
30036;BERNIS;GARD;LANGUEDOC-ROUSSILLON
72019;AVESSE;SARTHE;PAYS DE LA LOIRE
72087;CONFLANS-SUR-ANILLE;SARTHE;PAYS DE LA LOIRE
16347;SAINT-ROMAIN;CHARENTE;POITOU-CHARENTES
58108;EMPURY;NIEVRE;BOURGOGNE
54387;MORVILLE-SUR-SEILLE;MEURTHE-ET-MOSELLE;LORRAINE
22190;PLESLIN-TRIGAVOU;COTES-D'ARMOR;BRETAGNE
62604;NEUFCHATEL-HARDELOT;PAS-DE-CALAIS;NORD-PAS-DE-CALAIS
53039;LE BOURGNEUF-LA-FORET;MAYENNE;PAYS DE LA LOIRE
55397;PAGNY-LA-BLANCHE-COTE;MEUSE;LORRAINE
60645;TRIE-LA-VILLE;OISE;PICARDIE
39505;SAUGEOT;JURA;FRANCHE-COMTE
88100;CHEF-HAUT;VOSGES;LORRAINE
44131;PORNIC;LOIRE-ATLANTIQUE;PAYS DE LA LOIRE
```



JSON

```
{
  first_name: 'Paul',
  surname: 'Miller',
  city: 'London',
  location: {
            type : 'Point',
            coordinates : [45.123,47.232]
  },
  cars: [
    { model: 'Bentley',
      year: 1973,
      value: 100000, … },
    { model: 'Rolls Royce',
      year: 1965,
      value: 330000, … }
  ]
}
```



Columnar Big Data File Formats

- Free & Open Source.
- Increased query performance over row-based data stores.
- Provides efficient data compression.
- Designed for performance on large data sets.
- Supports limited schema evolution.
- Is a splittable "file format".
- A Column-Oriented data store

Row Format

ID	Name	Score
1	john	4.1
2	mike	3.5
3	sally	6.4

Column Format

ID:	1	2	3
Name:	john	mike	sally
Score:	4.1	3.5	6.4







ORC vs Parquet: Key Differences in a Nutshell

ORC (Optimized Row Columnar) and Parquet are two popular big data file formats. Parquet is generally better for write-once, read-many analytics, while ORC is more suitable for read-heavy operations. ORC is optimized for Hive data, while Parquet is considerably more efficient for querying. Both support complex data structures, multiple compression options, schema evolution, and data skipping

RDD Operations

An Action in Spark is any operation that does not return an RDD. Evaluation is executed when an action is taken. Actions trigger the scheduler, which build a directed acyclic graph (DAG) as a plan of execution. The plan of execution is created by working backward to define the series of steps required to produce the final distributed dataset (each Partition). A transformation are functions that return another RDD.

TRANSFORMATION

RDD → RDD

Such operations apply transformations and generate a new RDD out of an existing RDD.

ACTION

RDD → Value

Such operations generate a value out of an RDD. No processing happens until an action is performed.

Spark is Lazy

Transformations are lazy in nature, they get execute when we call an action. They are not executed immediately.

- Not forced to load all data at step.
- Easier to parallelize operations: N different transformations can be processed on a single data element, on a single thread, on a single machine.
- Most importantly, it allows the framework to automatically apply various optimizations.

Transformation

Transformations always return a DataFrame (or in some cases, such as Scala & Java, Dataset[Row]).

Transformations are operations on RDDs, DataFrames, or Datasets that produce a new distributed dataset from an existing one. They are generally lazy, meaning they are not executed immediately but create a logical execution plan.

Typed Transformations returns *Dataset[T]*	Untyped Transformations returns *DataFrame*
alias(..)	agg(..)
as(..)	apply(..)
coalesce(..)	col(..)
distinct(..)	crossJoin(..)
dropDuplicates(..)	cube(..)
except(..)	drop(..)
filter(..)	groupBy(..)
flatMap(..)	join(..)
groupByKey(..)	rollup(..)
intersect(..)	select(..)
join(..)	selectExpr(..)
limit(..)	stat(..)
map(..)	withColumn(..)
mapPartitions(..)	withColumnRenamed(..)
orderBy(..)	
randomSplit(..)	
repartition(..)	
sample(..)	
select(..)	
sort(..)	
sortWithinPartitions(..)	
union(..)	
where(…)	

Action

Transformations always return a DataFrame (or in some cases, such as Scala & Java, Dataset[Row]).

In contrast, Actions either return a result or write to disc. For example:

- The number of records in the case of count()
- An array of objects in the case of collect() or take(n)

Method	Return	Description
collect()	Collection	Returns an array that contains all of Rows in this Dataset.
count()	Long	Returns the number of rows in the Dataset.
first()	Row	Returns the first row.
foreach(f)	-	Applies a function f to all rows.
foreachPartition(f)	-	Applies a function f to each partition of this Dataset.
head()	Row	Returns the first row.
reduce(f)	Row	Reduces the elements of this Dataset using the specified binary function.
show(..)	-	Displays the top 20 rows of Dataset in a tabular form.
take(n)	Collection	Returns the first n rows in the Dataset.
toLocalIterator()	Iterator	Return an iterator that contains all of Rows in this Dataset.

Modify and save dataframes

- Load source file into a dataframe
- Use dataframe methods and functions to transform the data:
  - Filter rows
  - Modify column values
  - Derive new columns
  - Drop columns
- Write the modified data
  - Specify required file format

```
from pyspark.sql.functions import year, col

# Load data
df = spark.read.load("/data/orders.csv",
                     format="csv", header=True)

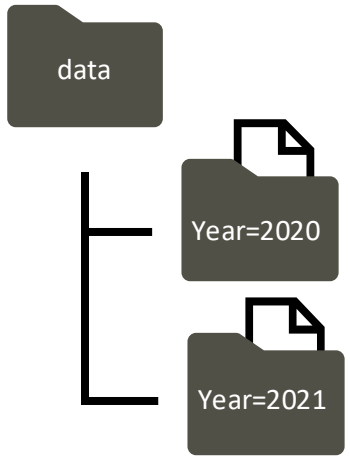
# Add Year column, derived from OrderDate
df = df.withColumn("Year", year(col("OrderDate")))

# Save transformed data
df.write.mode("overwrite").parquet("/data/orders.parquet")
```

Partition data files

- Partition data by one or more columns
- Distributes data to improve performance and scalability

```
df.write.partitionBy("Year").mode("overwrite").parquet("/data")
```



Transform data with SQL

- Use the metastore to define tables and views
- Use SQL to query and transform the data
- Save transformed data as an external table
  - Dropping an external table does not delete the data files

```
# Create a view in the metastore
df.createOrReplaceTempView("sales_orders")

# Use SQL to transform data and return a dataframe
new_df = spark.sql("SELECT OrderNo, OrderDate, Year(OrderDate) As Year FROM sales_orders")

# Save the dataframe as an external table
new_df.write.partitionBy("Year").saveAsTable("transformed_orders", format="parquet",
                                             mode="overwrite", path="/transformed_orders")
```

DE 0.00 - Module Introduction



DataFrames

Type	Lesson
Demo	DE 0.01 - Spark SQL
Lab	DE 0.02L - Spark SQL Lab
Demo	DE 0.03 - DataFrame & Column
Lab	DE 0.04L - Purchase Revenues Lab
Demo	DE 0.05 - Aggregation
Lab	DE 0.06L - Revenue by Traffic Lab

Functions

Type	Lesson
Demo	DE 0.07 - Reader & Writer
Lab	DE 0.08L - Ingesting Data Lab
Demo	DE 0.09 - Datetimes
Demo	DE 0.10 - Complex Types
Demo	DE 0.11 - Additional Functions
Lab	DE 0.12L - Abandoned Carts Lab

Query Execution

Type	Lesson
Demo	DE 0.13 - Query Optimization